

chapter5

제5장 데이터 중복 방지와 일관성 보장

4장 실습은 실제 브로커에서 두 가지 실패를 실측으로 남겼다. 커밋을 처리보다 앞세우면 메시지가 조용히 유실되고(유실 20건), 처리를 커밋보다 앞세우면 같은 메시지가 두 번 수신된다(중복 10건). 유실은 복구할 수 없지만 중복은 교정할 수 있으므로, 실무 기본값은 at-least-once를 선택하고 중복을 하류에서 제거하는 것이라고 했다. 이 장은 그 "하류 설계"를 다룬다. 먼저 중복이 Producer·브로커·Consumer·DB의 어느 단계에서 왜 발생하는지 분석하고, "무엇을 같은 이벤트로 볼 것인가"를 정하는 중복 제거 키를 설계한다. 이어서 저장소의 유니크 제약과 Upsert로 "최소 한 번 전달 + 정확히 한 번 반영"을 실제로 만들고, 4장이 남긴 실측 중복 스트림으로 검증한다(★★☆☆☆). 마지막으로 지연 이벤트와 watermark 개념을 열어 6장(스트리밍 처리)으로 연결한다.

학습 목표

분산 시스템에서 중복이 발생하는 원인을 단계별로 짚고, 각 단계의 대응을 설명할 수 있다.
업무 요구에 맞는 중복 제거 키와 저장소 제약 조건을 설계할 수 있다.
at-most-once, at-least-once, exactly-once의 차이를 실측 근거로 설명할 수 있다.

5.1 Producer, Broker, Consumer, DB 단계별 중복 원인

학습 목표

"같은 메시지가 두 번 왔다"는 증상의 원인 지점을 단계별로 구분한다.
전달 의미론 3종을 유실·중복·비용의 트레이드오프로 비교한다.

핵심 개념: 재전송은 중복 이벤트를 만들 수 있다

중복은 대부분 버그가 아니라, 유실을 막기 위해 재시도하거나 재처리할 때 발생한다. 메시지가 저장된 뒤 확인 응답만 유실되면 Producer는 같은 메시지를 다시 보내기 때문이다.

Producer 단계: 브로커가 메시지를 기록했지만 확인 응답(ack)이 유실되면, Producer는 "전송 실패"로 판단해 재시도한다. 브로커에는 같은 메시지가 두 번 기록된다. `acks=all`처럼 안전한 설정일수록 재시도는 더 적극적이다 — 멍든 Producer가 이를 브로커 수준에서 제거한다(보충).

브로커 단계: 브로커 자체는 받은 것을 저장할 뿐 중복을 생성하지 않는다. 다만 리더 교체 과정에서 Producer 재시도와 결합해 중복이 기록될 수 있다.

Consumer 단계: 4장 시나리오 C에서 실측한 경로다. 처리 후 커밋 전에 크래시(crash)하면, 재시작한 컨슈머는 옛 커밋 위치부터 다시 읽는다 — 40건의 처리 기록에 고유 이벤트는 30건, 재수신 중복이 10건이었다.

DB 단계: 소비 애플리케이션이 “저장 성공, 커밋 실패” 후 재시작하면 같은 이벤트를 다시 저장한다. 배치 재실행(7장), 리플레이 재처리(4.1절)도 같은 결과를 초래한다. **저장소에 방어선이 없으면 상류의 모든 중복이 데이터가 된다.**

Producer 중복의 발생 과정을 타임라인으로 정리하면 재시도가 왜 중복을 발생시키는지가 분명해진다.

시점	Producer	브로커
t1	민원 이벤트 전송	—
t2	(응답 대기)	로그에 기록, 확인 응답 발송
t3	확인 응답이 네트워크에서 유실	(이미 기록 완료)
t4	타임아웃 → 실패로 판단 → 재전송	같은 이벤트를 또 기록

t3에서 Producer가 아는 것은 “응답이 오지 않았다”뿐이다. 기록이 안 된 것인지(재전송해야 함) 응답만 사라진 것인지(재전송하면 중복) Producer는 구분할 수 없다. 재전송을 포기하면 유실을 감수해야 하므로, 유실 비용이 큰 시스템은 중복을 허용하고 저장 단계에서 중복을 제거한다.

이 t3의 문제는 이 실습에 국한되지 않고 상용 결제 시스템이 일상적으로 다루는 문제다. Stripe의 결제 API 설계 문서는 이 상황을 **모호한 실패(ambiguous failure)**라 부른다 — 연결이 메시지 교환 도중 끊기면 클라이언트는 요청이 처리됐는지 아닌지 알 수 없고, “재시도가 안전한지”조차 판단할 수 없다는 것이다(Brandur Leach, Stripe, 2017 — 결제 중복 방지의 직접 선례). 이 곤경에 대한 산업의 표준 답이 뒤 5.2에서 다룰 먹등성 키다: 클라이언트가 요청마다 고유 ID를 부여하면, 모호한 실패가 발생해도 같은 ID로 안전하게 재시도할 수 있고 서버가 중복을 흡수한다. 즉 “응답을 받지 못했다”를 안전하게 만드는 유일한 방법은 재전송을 없애는 것이 아니라 재전송을 무해하게 만드는 것이며, 이것이 이 장 전체의 설계 방향이다.

워크드 예제(worked example): 중복 이벤트 한 건의 처리 과정(실측)

4장 시나리오 C가 남긴 처리 기록에서 중복된 이벤트 하나(alo-000003)의 두 기록을 그대로 옮기면 다음과 같다.

기록	파티션	오프셋	처리 시각(consumed_at)
1회차(크래시 전)	0	0	01:59:20
2회차(재시작 후)	0	0	01:59:35

두 기록의 (파티션, 오프셋)이 **완전히 같다**는 점이 핵심이다. 브로커의 로그에 이 이벤트는 한 건뿐이고(파티션 0의 오프셋 0), 컨슈머가 같은 위치를 두 번 읽은 것이다 — 커밋 전에 중단되어 읽기 위치가 되돌아갔기 때문이다. 즉 이 중복은 Producer 재전송(브로커에 두 건)이 아니라

Consumer 재수신(브로커에 한 건, 읽기 두 번)이며, 15초의 시차는 4장에서 본 세션 타임아웃 (session timeout, 10초)과 재시작 대기의 흔적이다. 중복을 발견했을 때 "(파티션, 오프셋)이 같은가"를 확인하면 어느 단계의 중복인지 즉시 판별할 수 있다 — 표 5.1의 진단을 실제 데이터로 수행한 셈이다.

표 5.1 단계별 중복 원인과 대응(중복 시나리오별 대응표)

단계	중복 발생 시나리오	1차 대응	한계
Producer	ack 유실 → 재전송	막등 Producer(보충)	Producer 재시작·앱 차원 재전송은 차단 불가
Broker	리더 교체 + 재시도 결합	복제·막등성 설정	단독 대응 없음(수동적)
Consumer	커밋 전 크래시 → 재수신	커밋 시점 설계(4장)	at-least-once인 한 중복은 정의상 발생
DB	재처리·재실행·리플레이 재삽입	중복 제거 키 + 유니크 제약 + Upsert(이 장)	키 설계가 틀리면 무력

표의 핵심은 마지막 행이다. 상류(Producer·Consumer)의 대응은 각 단계의 중복만 줄일 뿐, 어느 것도 "최종 데이터에 중복이 없다"를 보장하지 못한다. 그 보장은 데이터가 최종적으로 놓이는 저장소에서 만들어야 하며, 그래서 이 장의 무게중심은 DB 단계에 있다.

각 단계의 대응이 실제로 작동하는지는 지표로 확인한다. Consumer 단에서 가장 흔히 사용하는 것이 **메시지 중복률** — 같은 메시지를 두 번 이상 받은 비율이다. at-least-once를 사용하는 한 이 값이 0%가 아닌 것은 정상이므로, 절대값이 아니라 변화를 관찰한다. 갑자기 상승하면 커밋 실패나 리밸런싱(rebalancing) 이상을 의심한다. 이 해석 방식은 공공과 민간이 같다.

전달 의미론 3종 정리

4장에서 커밋 시점으로 경험한 개념을 정의로 정리한다.

표 5.2 전달 의미론 비교

의미론	정의	유실	중복	비용	4장 실측
at-most-once	최대 한 번 전달	가능	없음	가장 낮음	유실 20건 (시나리오 B)
at-least-once	최소 한 번 전달	없음	가능	중간	중복 10건 (시나리오 C)
exactly-once	정확히 한 번 반영	없음	없음(반영 기준)	가장 높음	— (트랜잭션·막등성 필요, 보충)

두 정의는 우편 배달 방식에 정확히 대응한다. at-most-once는 수취인이 부재중이면 재방문하지 않는 일반 우편과 같다 — 재전송이 없으므로 편지가 두 통 오는 일은 없지만, 안 올 수는 있다 (전달이 안 될 수 있으나 중복은 없음). at-least-once는 받았다는 서명을 받을 때까지 다시 오는 등기 우편과 같다 — 반드시 오지만, 서명 기록이 엇갈리면 같은 편지가 두 번 올 수 있다(전달은 보장되나 중복이 생길 수 있음). 4장 실습의 커밋 시점 선택은 이 두 배달 정책 중 하나를 선택하는 것이었다 — 커밋을 처리보다 앞세우면 at-most-once, 처리를 커밋보다 앞세우면 at-least-once가 된다.

exactly-once에서 주의할 표현이 있다. 네트워크 세계에서 “정확히 한 번 전달”을 절대적으로 보장하기는 어렵다. 이것은 구현 실력의 문제가 아니라 원리적 한계에 가깝다 — 두 노드가 네트워크로 메시지를 주고받는 한, 발신자는 “수신자가 받았는가”를 확인 응답으로만 알 수 있고, 그 응답 자체가 또 유실될 수 있다(두 장군 문제, Two Generals Problem — ACK 불확실성의 고전적 사고실험). 분산 시스템 실무자들이 자주 인용하는 한 논쟁적 글은 이 점을 강하게 주장한다: “분산 시스템에서 exactly-once 전달은 있을 수 없다. 어떤 메시지 큐가 exactly-once를 표방한다면, 그것은 팔기 위한 과장이거나 분산 시스템을 이해하지 못한 것”이라는 강한 주장이다(Tyler Treat, 2015 — 배경 자료, 저자의 입장이 뚜렷한 블로그 글). 그렇다면 실무의 exactly-once는 무엇인가. 같은 글에 답도 제시되어 있다 — “실무에서 exactly-once를 달성하는 방법은 **홍내 내는 것이다**: 메시지 자체를 멍등하게 만들거나, 중복 제거로 멍등성의 필요를 없앤다.” 즉 실무의 exactly-once는 대개 “여러 번 전달되어도 **정확히 한 번 반영**된 것과 같은 결과”를 뜻하며, 그 반영을 만드는 소비 측 기법이 바로 이 장의 멍등 쓰기(idempotent write)와 중복 제거다. 낱품 문서에서 “exactly-once 지원”이라는 문구를 확인하면, 그것이 전달 보장인지 반영 보장인지를 되묻는 것이 검수의 출발점이 되는 이유다.

멍등(idempotent)은 같은 연산을 여러 번 적용해도 결과가 한 번 적용한 것과 같은 성질이다 — 엘리베이터 버튼처럼, 이미 눌린 버튼을 다섯 번 더 눌러도 호출은 한 번인 것과 같은 구조다(반복 입력이 단일 효과로 수렴한다는 속성이 정확히 대응한다). 반대의 예가 “잔액 += 입금액” 같은 누적 연산이다: 같은 입금 이벤트가 두 번 반영되면 잔액이 잘못된 값이 된다. 중복 제거의 목표를 한 문장으로 요약하면 **모든 쓰기를 멍등 연산으로 만드는 것이다**. “이벤트 X를 반영하라”가 이미 반영된 상태에서 또 와도 상태가 변하지 않는다면, at-least-once 전달 위에서 exactly-once 반영을 얻는다.

민간의 중복 방지를 공공으로 옮기기

중복 제거는 흔히 광고·커머스·결제 같은 민간 대용량 시스템의 기술로 소개되지만, 동일한 처리 절차를 공공에도 적용할 수 있다. 민간에서는 광고 노출·클릭 이벤트의 재전송 중복을 제거하고, 결제 API의 재시도에도 카드가 한 번만 청구되도록 멍등성 키를 사용한다. 공공에서는 민원 통계의 중복 집계, 보조금의 이중 지급, 재난 문자의 중복 발송을 막는 데 같은 절차를 적용한다.

표 5.3 민간 중복 방지 기법의 공공 적용

민간 현장	무엇을 막는가	공공 대응물	공공 특수성에 따른 변형
결제 API 멍등성 키	카드 이중 청구	보조금·환급 이중 지급 방지	지급은 되돌리기 어려우므로 “발생 전 차단”이 원칙
광고 이벤트 dedup	노출·클릭 과다 집계	민원 이중 접수로 인한 통계 왜곡 방지	제거한 중복도 “계수해” 감사에 남긴다 (seen_count)
알림 재전송 억제	푸시 중복 발송	재난 문자 중복 발송 방지	발송은 부작용이라 발송 전 유니크 제약으로 거른다

셋째 열이 같아 보여도 넷째 열에서 공공과 민간의 차이가 나타난다. 민간에서 중복은 대개 **제거하면 되는 잡음**이지만, 공공에서 중복은 **계수해 보고해야 하는 사건**이다. “이번 달 민원이 왜 줄었는가”라는 감사 질문에 “중복 12건을 제거했기 때문”이라고 답하려면, 제거한 중복의 수가 어딘가에 남아 있어야 한다. 그래서 공공의 중복 제거는 “제거했다”가 아니라 “계수해 반영하지 않았다”의 형태를 띠며, 이 차이가 5.3의 병합(UPSERT·seen_count) 설계를 공공에서 특히 중요하게 만든다.

검수 포인트(이 절에서 도출되는 질문)

이 시스템의 전달 의미론은 무엇으로 문서화되어 있는가 — “Kafka를 사용한다”는 답은 답이 아니다(커밋 시점·재시도 설정이 답이다).

각 단계(Producer/Consumer/DB)의 중복 대응이 표 5.1처럼 명시되어 있는가, 아니면 “어딘가에서 걸러지겠지”인가.

최종 저장소에 중복 제거 키와 유니크 제약이 실제 스키마로 존재하는가 — 설계 문서가 아니라 DDL로 확인한다.

공공 사례

민원 통계에서 중복은 곧 왜곡이다. 같은 민원이 두 번 집계되면 “이번 달 불법주차 민원 12% 증가” 같은 수치의 신뢰성이 낮아지고, 이 수치가 인력 배치·예산 근거로 사용된다면 중복 10건은 단순한 기술 문제가 아니다. 감사에서 “이 수치는 중복이 제거된 값인가”라는 질문에 답하려면, 어느 단계에서 어떤 방어선이 작동하는지 표 5.1처럼 문서화되어 있어야 한다. - 중복은 재시도·재처리라는 유실 방지 장치의 부작용으로, 단계마다 발생 경로가 다르다. - (파티션, 오프셋)의 일치 여부로 재수신 중복과 재전송 중복을 판별할 수 있다 — 실측으로 확인했다. - 상류 대응은 중복을 줄일 뿐이고, “최종 데이터의 무중복” 보장은 저장소에서 만든다. - exactly-once는 실무적으로 “정확히 한 번 반영”이며, 멍등 쓰기로 달성한다.

5.2 중복 제거 키 설계

학습 목표

중복 제거 키가 “무엇을 같은 이벤트로 볼 것인가”의 업무 정의임을 이해한다.
키 후보를 안정성·유일성 기준으로 비교해 선택한다.

핵심 개념

저장소가 중복을 제거하려면 “이 두 행은 같은 이벤트다”를 판정할 기준이 필요하다. 그 기준이 **중복 제거 키(deduplication key)** 다. 키 선택은 기술이 아니라 업무 정의의 문제다 — 같은 민원인이 같은 내용을 이틀 연속 접수하면 중복인가, 별건인가? 답은 시스템이 아니라 업무 규정이 정한다.

키 후보는 성격이 다르다.

생성 시점의 고유 ID: 이벤트를 만드는 쪽이 발급하는 ID(4장 실습의 `event_id` 같은). 발급 이후의 모든 재전송·재처리에서 동일하게 유지되므로 기술적 중복(재시도·재수신) 제거에 가장 확실하다. 단, ID가 **재시도보다 먼저** 만들어져야 한다 — 전송 시마다 새 ID를 발급하면 재전송이 서로 다른 이벤트가 되어 키가 무력해진다.

업무 식별자: 민원 접수번호처럼 업무 시스템이 부여하는 번호. 업무적 의미가 분명하지만, 접수번호가 발급되기 전 단계의 이벤트에는 쓸 수 없고, 시스템 간 형식이 다를 수 있다.

내용 해시: 이벤트의 주요 필드를 이어 붙여 해시한 값. 원천이 ID를 주지 않을 때의 차선책이다. 그러나 “같은 내용 = 같은 이벤트”라는 가정이 업무적으로 옳은지 먼저 확인해야 한다 — 같은 위치의 불법주차 민원이 한 시간 간격으로 두 번 접수된 것은 다른 이벤트일 수 있다.

표 5.4 민원 이벤트의 중복 제거 키 후보 비교(워크드 예제)

후보	기술적 중복 제거	업무적 중복 판정	위험
생성 시점 고유 ID	확실	못 함(재접수는 별개 ID)	발급 시점이 재시도 이후면 무력
업무 식별자(접수번호)	가능	가능	발급 전 이벤트에 부재
내용 해시	가능	위험(별건을 중복으로 오판)	필드 하나만 달라져도 다른 키

표를 해석하는 기준은 열의 구분이다. “기술적 중복 제거”는 4장에서 본 재시도·재수신을 거르는 능력이고, “업무적 중복 판정”은 사람의 행위(재접수)를 묶는 능력이다. 어떤 후보도 두 열을 동시에 충족하지 못한다는 것이 이 표의 결론이며, 그래서 다음 문장이 나온다.

실무 정석은 **기술적 중복(재시도·재수신)은 생성 시점 고유 ID로 제거하고, 업무적 중복(재접수 등)은 별도 규칙으로 다루는 것이다**. 두 층위를 한 키로 해결하려는 시도가 흔한 실패 원인이다.

“생성 시점”이 중요한 이유

같은 UUID라도 발급 위치에 따라 결과가 달라진다. 타임라인으로 정리하면 다음과 같다.

발급 위치	이벤트 생성 시	전송 함수 안에서
정상 전송	ID = X 발급 → 전송	전송 직전 ID = X 발급 → 전송
재전송	같은 X로 재전송	전송 직전 새 ID = Y 발급 → 재전송
저장소에서	X 충돌 → 중복 제거됨	X와 Y는 다른 행 → 중복 통과

오른쪽 열은 코드만 보면 “모든 이벤트에 고유 ID가 있다”는 요구를 완전히 만족한다 — 그런데 중복 제거는 수행하지 못한다. 검수에서 “이벤트에 고유 ID가 있는가”만 묻고 “그 ID가 재시도 경로에서 불변인가”를 묻지 않으면 이 결함을 통과시키게 된다. 4장 실습의 Producer가 이벤트를 생성할 때 (`make_event`) ID를 부여하고 전송 루프에서는 변경하지 않는 것이 바로 왼쪽 열의 구현이다.

덧붙여, 이 장 실습의 중복 제거가 성립한 전제도 같은 것이다 — `alo-000003`이라는 ID가 크래시와 재시작을 관통해 불변이었기에, 저장소가 두 기록을 같은 이벤트로 판정할 수 있었다.

산업 표준: Stripe의 멍등성 키(직접 선례)

“생성 시점에 발급되어 재시도 경로에서 불변인 고유 ID”는 이 교재가 고안한 원칙이 아니라, 결제 같은 고위험 도메인에서 이미 표준으로 정착한 설계다. Stripe 결제 API는 이를 **멍등성 키(idempotency key)**로 구현한다 — 클라이언트가 요청 하나마다 고유 값을 만들어 `Idempotency-Key` 헤더에 실어 보내고, 네트워크 오류로 실패하면 **같은 키로** 재시도한다. 서버는 그 키를 처음 수신하면 정상 처리하고, 이미 수신한 키면 앞선 결과를 그대로 반환한다. 그 결과가 문서의 핵심 문장이다 — “네트워크 연결 오류로 요청이 실패해도 같은 멍등성 키로 안전하게 재시도할 수 있고, 고객은 **한 번만 청구된다**”(Brandur Leach, Stripe, 2017).

세 가지가 이 장의 원칙과 정확히 일치한다. 첫째, 키는 **클라이언트가**(즉 이벤트 생성 시점에) 발급한다 — 5.2의 “생성 시점” 요건 그대로다. 둘째, 재시도는 **같은 키**를 재사용한다 — 앞의 발급 위치 타임라인에서 오른쪽 열(전송마다 새 ID)이 왜 무력한지를 산업이 같은 이유로 배격한 것이다. 셋째, 서버(저장소)가 최종 판정을 내린다 — 방어선이 클라이언트가 아니라 서버에 있다는 5.3의 명제와 같다. 여기까지가 Stripe 문서가 직접 다루는 결제 도메인의 내용이다(직접 선례). 공공으로 옮길 때는 자격을 낮춰야 한다 — 카드 이중 청구 방지와 보조금·환급 이중 지급 방지는 대상과 규제가 달라 “그대로 이식”이 아니라 **방법론 차용**이다. 그럼에도 핵심 구조(생성 시점 고유 키 + 재시도 불변 + 서버 판정)는 그대로 적용된다는 것, 그리고 그 구조가 결제처럼 실패 비용이 큰 도메인에서 이미 표준으로 정착했다는 것이 이 선례의 실익이다.

결제 현장은 이 방어선을 지표로도 관찰한다. 멍등성 키에 걸려 차단된 이중 청구의 비율을 **결제 중복 방지율**로 두고, 이 값이 갑자기 상승하면 상류 게이트웨이(gateway)에 재시도가 집중되었다고 해석한다. 공공에서도 같은 해석이 성립한다 — 보조금 이중 지급 차단 건수가 급증하면 원천 시스템의 재전송을 먼저 점검한다.

워크드 예제: 재접수는 어느 층에서 잡는가

같은 시민이 같은 불법주차를 오늘 접수하고, 처리되지 않았다는 이유로 다음 날 다시 접수했다고 가정한다. 두 접수는 각각 생성 시점에 서로 다른 event_id를 받는다 — 기술적으로는 완벽히 별개의 이벤트이고, 유니크 제약을 아무 문제 없이 통과해 두 행이 된다. 이것은 결함이 아니라 **올바른 동작**이다: 두 번의 접수 행위가 실제로 있었고, 감사 관점에서는 둘 다 기록되어야 한다. “사실상 같은 민원이니 묶어서 처리하자”는 판단은 그 위의 업무 규칙 계층(예: 같은 위치·같은 유형·N일 이내 → 병합 검토 대상으로 표시)에서, 자동 삭제가 아니라 **표시와 사람의 확인**으로 다루는 것이 안전하다. 기술 키가 지우는 것은 “같은 이벤트의 반복 전달”이지 “비슷한 사건의 반복 발생”이 아니다 — 이 구분이 성립하지 않으면 시민의 정당한 재민원이 시스템에서 기록 없이 유실된다.

이 키 층위 분리가 실제 통합 시스템에서 어떻게 작동하는지는 14장의 장애 드릴에서 실측으로 확인된다. 14장 통합 MVP는 업무 식별자(민원 접수번호)를 UPSERT 키로 삼는데, 재전송 드릴에서 7/2 이벤트의 마지막 10건을 한 번 더 발행하자 물리 수신 28건이 고유 17건으로 축약되고 매핑 집계는 7장 확정치와 같은 16건으로 유지되었다 — 흡수된 11건의 내역은 다음과 같다. 10건은 수송 계층의 재전송(ack 유실 후 재발행)이었고 1건은 원천 파일 안에 같은 접수번호가 두 번 들어온 업무 계층의 중복이었는데, **소비자 관점에서는 동일하게 보인다**(같은 접수번호의 재도착). 키가 접수번호이므로 원인을 구분할 필요 없이 둘 다 흡수되고 seen_count에 관찰 흔적만 남았다. 여기서 흡수된 “업무 중복 1건”은 5.2가 통과시켜야 한다고 한 재민원과 모순되지 않는다 — 그것은 **같은 접수번호의 물리적 재도착**이지 새 접수번호를 받은 별건 재민원이 아니기 때문이다. 키가 “같은 이벤트”의 경계를 어디에 긋느냐가 무엇이 흡수되고 무엇이 통과하는지를 결정한다는 것, 그 경계가 곧 업무 정의라는 것을 14장의 실측에서 확인할 수 있다.

키 설계 점검 체크리스트(검수 자산)

중복 제거 키를 확정하기 전에 다음을 순서대로 점검한다.

- ☐ “같은 이벤트”의 업무 정의가 문서로 합의되어 있는가(재접수는 별건인가)
- ☐ 키(ID)는 이벤트 **생성 시점**에 발급되는가 — 전송·재시도 코드 안에서 발급되지 않는가
- ☐ 키는 재시도·재수신·리플레이 경로 전체에서 불변인가
- ☐ 키의 유일성 범위는 충분한가(시스템 재기동·연도 경과 후에도 충돌하지 않는가)
- ☐ 업무적 중복(재접수 등)을 다룰 별도 규칙 계층이 지정되어 있는가

공공 사례

기관 간 데이터 연계에서 키 설계는 더 어려워진다. A 기관의 민원 ID와 B 기관의 이송 접수번호가 같은 사건을 가리켜도 형식이 다르면 기계적으로는 별건이다. 공공 데이터 표준화 논의에서 식별자 체계가 반복적으로 등장하는 이유이며, 토픽 설계표(4장 표 4.2)에 키 정의를 문서화해 두면 연계 시점에 판정 근거가 된다. - 중복 제거 키는 “같은 이벤트”의 정의이며, 업무 규정과 함께 설계한다. - 기술적 중복은 생성 시점 고유 ID로, 업무적 중복은 별도 규칙으로 — 층위를 분리한다. - 키는 재시도·재처리 경로 전체에서 불변이어야 한다.

5.3 저장소 Upsert와 유니크 제약

학습 목표

유니크 제약이 왜 “최후의 방어선”인지 이해한다.
충돌 처리 3방식(실패·무시·병합)의 선택 기준을 판단한다.

핵심 개념

중복 제거 키를 정했으면, 저장소에 그 키의 **유니크 제약(unique constraint)** 을 건다. 흔한 대안 — 애플리케이션 코드에서 “저장 전에 존재를 확인하고 없으면 저장” — 이 왜 부족한지부터 확인한다. 컨슈머 두 대가 같은 이벤트(중복 전달)를 거의 동시에 처리하는 타임라인이다.

시점	컨슈머 1	컨슈머 2
t1	SELECT: 이벤트 X 없음	—
t2	—	SELECT: 이벤트 X 없음 (아직 아무도 안 넣음)
t3	INSERT X — 성공	—
t4	—	INSERT X — 성공 → 중복 두 행

두 컨슈머 모두 “확인 후 저장”이라는 규칙을 준수했는데 중복이 발생했다. 확인(t1, t2)과 저장(t3, t4) 사이의 간격에 다른 컨슈머의 동작이 삽입될 수 있기 때문이다 — 이런 실패를 경쟁 조건(race condition)이라 부른다. 반면 유니크 제약이 있으면 t4의 INSERT는 저장소 안에서 원자적으로(확인과 기록이 분리되지 않게) 거부된다. 중복 방지 규칙을 애플리케이션 코드가 아니라 스키마 제약으로 구현하는 이유다: 코드는 인스턴스 수만큼 복제되어 서로를 모르지만, 제약은 데이터가 놓이는 단 한 곳에서 적용된다.

제약에 걸린 INSERT(충돌)를 어떻게 처리할지는 세 방식이 있다.

실패(기본): 충돌 시 오류를 발생시킨다. 중복이 “있어서는 안 되는 이상 상황”일 때 — 오류로 드러내고 조사한다.

무시(INSERT OR IGNORE): 충돌 행을 조용히 건너뛴다. 재전송이 항상 같은 내용일 때 (순수한 기술적 중복) 적합하다.

병합(UPSERT, ON CONFLICT DO UPDATE): 충돌 시 기존 행을 갱신한다. 재전송이 새 정보를 담을 수 있을 때(상태 갱신, 최신 관찰 시각), 또는 중복 발생 자체를 기록하고 싶을 때 적합하다.

표 5.5 INSERT 충돌 처리 3방식 비교

방식	충돌 시 동작	중복의 흔적	적합한 상황
실패(기본)	오류 발생, 적재 중단	오류 로그로 남음	중복이 이상 신호인 시스템(조사 필요)
무시(OR IGNORE)	행을 건너뛴	남지 않음	재전송이 항상 동일 내용(기술적 중복)
병합(UPSERT)	기존 행 갱신	갱신 필드로 남음	재전송이 갱신을 담거나, 발생 자체가 정보

선택 기준은 하나의 질문으로 정리된다: “**두 번째 도착이 새 정보를 담고 있는가?**” 담고 있지 않으면 무시로 충분하고, 담고 있거나 도착 사실 자체가 정보라면 병합이다. 결정 흐름으로 정리하면 다음과 같다.

이 테이블에서 중복 도착은 정상 상황인가? — 아니오(이상 신호)라면 → **실패**(오류로 드러내고 조사)
 정상이라면, 재전송이 갱신을 담을 수 있는가? — 예(상태 변경 등)라면 → **병합**
 항상 동일 내용이라면, 중복 발생 빈도를 지표로 사용하는가? — 예라면 → **병합**(seen_count),
 아니오라면 → **무시**

구현과 실행

세 방식이 SQL에서 어떻게 다른지 나란히 비교한다(모두 event_id에 유니크 제약이 있다는 전제).

방식	SQL	충돌 시
실패	<code>INSERT INTO t VALUES (...)</code>	오류 발생, 트랜잭션 중단
무시	<code>INSERT OR IGNORE INTO t VALUES (...)</code>	아무 일 없음(rowcount 0)
병합	<code>INSERT INTO t VALUES (...) ON CONFLICT(event_id) DO UPDATE SET ...</code>	지정 필드만 갱신

실습은 SQLite(파이썬 표준 라이브러리)를 사용한다. 병합 방식의 핵심은 다음과 같다 — 처음 도착한 이벤트는 행이 되고, 다시 도착한 이벤트는 재관찰 기록(seen_count, last_seen_at)이 된다.

```
INSERT INTO complaints_upsert VALUES (?, ?, ?, ?, ?, 1)
ON CONFLICT(event_id) DO UPDATE SET
    last_seen_at = excluded.last_seen_at, seen_count = seen_count + 1
```

전체 코드는 [practice/chapter4/code/4-3-deduplicate-events.py](#) 참고

SQL을 한 줄씩 검토한다. `INSERT ... VALUES (... , 1)` 은 “처음 수신한 이벤트면 행을 생성하고 관찰 횟수를 1로 시작하라”다. `ON CONFLICT(event_id)` 는 “event_id 유니크 제약에 걸리면(= 이미 수신한 이벤트면)”이고, `DO UPDATE SET` 이 그때의 동작이다. `excluded` 는 SQLite의 예약 이름으로 **충돌을 일으킨 새 행**(삽입되려다 거부된 값)을 가리킨다 — 그래서 `last_seen_at = excluded.last_seen_at` 은 “마지막 관찰 시각을 방금 도착한 것의 시각으로 갱신”이 된다. 반면 `first_seen_at` 은 `DO UPDATE` 목록에 없으므로 최초 삽입값이 영원히 보존된다 — 무엇을 갱신하고 무엇을 보존할지가 SQL 한 줄에 명시적으로 드러나는 것이 UPSERT의 장점이다.

이 설계는 중복을 버리지 않고 **관찰 데이터로 바꾼다**. `seen_count > 1` 인 행의 수는 “같은 이벤트가 두 번 이상 도착한 이벤트 수”이므로, 상류 파이프라인의 건강 상태(재시도·재처리 빈도)를 저장소에서 역으로 관찰할 수 있다. 단 한 가지 조건을 붙여야 한다 — 이 수치를 “장애로 인한 재수신”으로 직접 해석하려면, 장애 재수신과 계획된 리플레이(정기 재처리)를 구분할 원인 필드가 함께 있어야 한다. 둘을 혼합해 계수하면 계획 리플레이까지 장애로 오인해 지표가 과대 산출되기 때문이다(이 구분은 실습 D 단계에서 실측으로 다시 확인한다).

이 관찰값에 이름을 붙이면 **Upsert 충돌률**이다 — 유니크 제약에 걸려 병합된 행의 비율이며, `seen_count > 1` 인 행의 비율과 같은 값이다. 계산식은 하나인데 용도가 도메인마다 다르다. 민간에서는 이 값을 대시보드에 올려 SLA(Service Level Agreement, 서비스 수준 약정) 준수와 장애 조기 탐지에 활용한다. 공공에서는 같은 값이 감사 답변의 출처가 된다 — “이번 달 중복 12건을 제거했다”고 말하려면 그 12를 어디서 계수했는지 제시해야 하기 때문이다.

실습의 SQLite가 운영 DB로 옮겨지는 이유

실습이 SQLite를 사용하지만 여기서 학습한 내용은 실습용에 국한되지 않는다. 유니크 제약과 UPSERT는 특정 제품만의 기능이 아니라 주요 관계형 DB가 공통으로 지원하는 실무 표준 패턴이기 때문이다. 운영에서 가장 많이 사용되는 PostgreSQL도 같은 구조를 제공한다 — `INSERT ... ON CONFLICT (event_id) DO UPDATE SET ...` 이며, SQLite의 `excluded` 에 해당하는 것이 Postgres에서는 `EXCLUDED` 다(PostgreSQL 공식 문서, `INSERT — ON CONFLICT`). 다만 한 가지는 정확히 구분해야 한다: `ON CONFLICT` 는 SQL 표준이 아니라 SQLite·PostgreSQL이 공유하는 확장 문법이고, ISO SQL 표준에 더 가까운 것은 `MERGE` 문이다(Oracle·SQL Server·최신 PostgreSQL이 지원). 즉 실습에서 만든 세 방식(실패·무시·병합)은 DB별 문법 차이(예: `ON CONFLICT` vs `MERGE` , `excluded` vs `EXCLUDED`)만 흡수하면 운영 DB로 거의 그대로 이식된다. 이것이 방어선을 애플리케이션 코드가 아니라 **저장소 제약**에 두는 또 하나의 이유다 — 코드는 언어·프레임워크를 바꾸면 다시 작성해야 하지만, 제약 기반 무결성(constraint-based integrity)은 DB를 옮겨도 같은 원리로 지켜진다. 검수자가 “중복 방어가 어디 있는가”를 물을 때 가장 강한 답은 애플리케이션 코드 라인이 아니라 DDL의 `UNIQUE · PRIMARY KEY` 한 줄인 이유이기도 하다.

민간→공공: 재난 문자와 “발송은 되돌릴 수 없다”

충돌 처리 방식 선택이 공공에서 특히 되돌릴 수 없어지는 사례가 재난 문자 발송이다. 민간의 푸시 알림 중복 억제와 목적은 같지만, 공공 재난 문자는 한 번 나가면 **회수가 불가능**하다는 점이 다르다 — 같은 경보가 두 번 발송되면 시민의 신뢰가 낮아지고, 최악의 경우 “시스템이 오작동한다”는

인식으로 실제 경보까지 무시된다(경보 피로). 여기서 발송은 “잔액 += 입금”처럼 **부작용을 가진 비역등 연산**이므로, 사후에 중복을 세는 것으로는 부족하고 **발송 전에** 차단해야 한다. 설계의 출발점은 “(재난 ID, 수신자) 쌍”에 유니크 제약을 걸어 같은 조합의 발송 시도를 저장소가 원자적으로 한 번만 통과시키는 것이다. 다만 여기에는 주의점이 하나 있다 — “INSERT 성공 시 곧바로 발송”으로 구현하면, INSERT 뒤 실제 발송 직전에 크래시했을 때 그 조합이 “이미 발송함”으로 잘못 기록되어 재시도에서 **발송이 누락된다**(외부 부작용은 트랜잭션 밖이라 DB 성공과 발송 성공이 원자적으로 묶이지 않는다). 그래서 실무는 유니크 제약에 **상태 컬럼(pending·sent·failed)**을 추가한 발송 아웃박스(outbox) 패턴을 사용한다: 먼저 (재난 ID, 수신자)를 `pending`으로 INSERT하고(충돌하면 이미 처리 중/완료이므로 건너뛴), 발송에 성공한 뒤에야 `sent`로 갱신하며, 발송 사업자 API가 역등성 키를 지원하면 그 키까지 함께 전달해 사업자 쪽 재시도 중복도 차단한다. 크래시로 `pending`에 남은 행은 재시도 배치가 다시 조회해 발송한다. 5.3의 세 방식으로 말하면, 최초 등록은 무시(OR IGNORE)에 가깝고 상태 전이·시도 횟수 기록은 병합(UPSERT:seen_count)으로 남기는 이중 구조가 된다 — “중복은 막되 누락은 만들지 않는다”가 공공 발송 설계의 핵심이다. 민간이 “중복 발송을 줄인다”의 최적화라면, 공공은 “중복을 보내면 안 된다 + 시도 이력은 남겨야 한다”의 규범이라는 차이로 인해 설계가 달라진다.

공공 사례

민원 상태 스트림(접수→배정→처리중→완료)을 “민원별 현재 상태” 테이블로 만드는 경우를 예로 든다. 키는 민원 건 ID, 값은 최신 상태다. 여기서 무시(OR IGNORE)를 선택하면 최초 상태(접수)로 고정되어 이후 갱신이 모두 폐기된다 — 명백히 병합(UPSERT)의 자리다. 반대로 “민원 접수 원장”처럼 접수 사실 자체를 기록하는 테이블이라면 재전송은 항상 같은 내용이므로 무시로 충분하다. 같은 시스템 안에서도 테이블의 역할에 따라 답이 달라지며, 그래서 충돌 처리 방식은 테이블 설계 문서에 개별적으로 명시되어야 한다. - 유니크 제약은 동시 실행에도 뚫리지 않는, 스키마 수준의 최후 방어선이다. - “확인 후 저장” 코드는 경쟁 조건에 뚫린다 — 방어선은 코드가 아니라 스키마에 둔다. - 충돌 처리는 실패·무시·병합 중 “두 번째 도착이 새 정보인가”로 선택한다. - 병합은 중복을 버리는 대신 파이프라인 건강 지표로 바꿀 수 있다.

5.4 스트리밍 처리의 watermark와 지연 이벤트

학습 목표

지연 이벤트가 집계계의 일관성을 어떻게 저해하는지 이해한다.
watermark가 “언제까지 기다릴 것인가”의 선언임을 이해한다.

핵심 개념

중복 제거로 “같은 이벤트가 두 번”은 막았지만, 일관성을 저해하는 문제가 하나 더 남아 있다. 2장에서 구분한 **이벤트 시간**(사건이 발생한 시각)과 **처리 시간**(시스템에 도착한 시각)의 차이, 즉 **지연 이벤트(late event)** 다.

23:00~24:00 구간의 민원 건수를 집계한다고 가정한다. 23:58에 발생한 민원이 네트워크 지연으로 00:03에 도착하면, 집계를 00:00에 확정된 시스템에서 이 이벤트는 반영될 자리가 없다. 버리면 집계는 틀리고, 받아서 재계산하면 “확정된 수치가 바뀌는” 문제가 생긴다 — 이미 보고된 통계가 소급 변경되는 것은 공공 보고에서 특히 민감하다.

watermark는 이 긴장에 대한 선언적 타협이다: “이벤트 시간 기준으로 X분까지의 지연은 기다려서 반영하고, 그보다 늦은 이벤트는 별도로 처리한다.” 앞의 예로 계산하면 — 23:58 발생·00:03 도착이면 지연은 5분이다. 지연 허용을 10분으로 선언했다면 이 민원은 기다림의 범위 안이므로 23시대 집계로 반영되고, 허용을 2분으로 선언했다면 반영되지 못한다. 즉 선언값 하나가 같은 이벤트의 반영 여부를 결정한다.

watermark를 길게 설정하면 정확하지만 확정이 늦어지고(상태를 오래 유지해야 한다), 짧게 설정하면 빠르지만 늦은 이벤트를 놓친다. 이것은 4장의 acks, 이 장의 충돌 처리처럼 또 하나의 **운영 트레이드오프 선언**이며, 값은 데이터의 실제 지연 분포를 관찰해 정한다.

이 “언제까지 기다릴 것인가”는 이 장에 세 번 나타난 같은 질문의 세 번째 사례다. 5.2에서 본 Stripe의 멍등성 키는 영구 보관되지 않는다 — 공개 문서는 키가 24시간 이상 지나면 시스템에서 자동 정리될 수 있다고 밝힌다. 정리된 뒤 같은 키로 재시도가 오면 서버는 그것을 처음 수신한 요청으로 취급한다. 보충에서 다룬 Redis 기반 dedup 캐시의 TTL도 동일하다 — “얼마나 오래 기억해야 중복을 걸러낼 수 있는가”의 문제다. 세 경우 모두 답의 형태가 같다: 기억(또는 대기)하는 시간창이 **중복·지연이 실제로 발생하는 시간창보다 넉넉해야** 방어가 성립하고, 짧으면 창 밖에서 도착한 것이 방어를 뚫는다. 재시도 최대 지연이 25시간인데 멍등성 키를 24시간만 보관하면 그 1시간의 간격에서 이중 청구가 발생하는 식이다. watermark의 지연 허용값을 데이터의 지연 분포로 정하듯, dedup의 보관 기간도 재시도·재수신의 실제 시간 분포로 정해야 하는 이유다.

지연 이벤트는 중복 문제와도 연결된다. 지연분을 반영하는 재계산은 같은 구간을 다시 집계하는 것이므로, 집계 결과 저장에도 5.3의 Upsert가 그대로 필요하다. 예를 들어 “23시대 민원 12건”이라는 집계 행이 이미 저장돼 있는데 지연 민원 1건이 반영되어 재계산되면, 새 결과는 “23시대 13건”이다. 이때 저장소가 naive INSERT면 23시대 행이 두 개(12와 13)가 되어 어느 쪽이 맞는지 알 수 없어진다. 집계 구간(“23시대”)을 키로 한 Upsert면 12가 13으로 덮어써져 항상 최신 재계산이 남는다 — 이벤트 저장에 쓴 방법이 집계 저장에도 똑같이 적용되는 것이다. 개념은 여기까지 두고, 윈도우 집계와 watermark의 구현은 6장에서 다룬다.

공공 사례

공공 통계의 “잠정치 → 확정치” 관행이 정확히 이 구조다. 월간 고용 통계가 잠정치로 먼저 나오고 이후 확정치로 갱신되는 것은, 지연 자료를 기다리는 기간(watermark에 해당)과 빠른 공표 사이의 제도화된 타협이다. 민간도 다르지 않다 — 광고·커머스의 실시간 대시보드는 늦게 도착한 전환·환불 이벤트로 매출 수치를 소급 갱신하며 잠정치와 확정치를 구분한다. 공공과 민간이 같은 구조를 사용하되, 공공은 확정치의 소급 변경이 “이미 보고된 통계의 정정”이라 절차적 무게가 훨씬 크다는 점만 다르다. 실시간 파이프라인 설계는 이 관행을 분 단위로 축소한 것이라 볼 수 있다 — 잠정치(창이 열려 있는 동안의 수치)와 확정치(창이 닫힌 후의 수치)를 구분해 표기하고, 확정치의 소급 변경은 예외 절차로 다룬다. “대시보드 수치가 이전과 다른 이유는 무엇인가”라는 질문에 대한 답은 “잠정치였기 때문”이며, 그 구분이 화면에 표시되어야 신뢰가 유지된다.

지연 이벤트를 다루는 시스템이라면 최소한 다음 지표를 운영 화면에 두어야 한다.

이벤트별 지연(발생 시각 대비 도착 시각)의 분포 — watermark 값의 근거
watermark 밖으로 밀려나 폐기된 이벤트 수 — 선언값이 현실과 맞는지의 신호
잠정치가 확정치로 바뀌기까지의 시간 — 보고 지연의 실측
확정 후 도착한 지연 이벤트 수 — 소급 정정 절차의 발동 빈도

역방향: 이 장의 설계를 민간 현장에 옮기면

지금까지는 민간의 중복 방지 기법을 공공으로 옮겼다. 반대 방향도 성립한다. 이 장에서 만든 설계를 그대로 민간 현장에 적용하면 무엇이 유지되고 무엇이 바뀌는지 세 현장에서 확인한다.

이커머스 재고 정합에서는 주문 이벤트가 재전송되면 재고 차감이 두 번 일어난다. 그러면 창고에 남아 있는 물건이 “재고 부족”으로 표시되어 판매가 중단된다. 대응 순서는 이 장 그대로다 — 주문 ID를 중복 제거 키로 삼고, 재고 차감 테이블에 유니크 제약을 걸고, 충돌은 병합으로 처리해 `seen_count` 를 남긴다. 같은 주문 ID로 재전송된 요청의 비율을 여기서는 주문 맥등성 키 충돌률이라 부르는데, 5.3의 Upsert 충돌률과 같은 값이다.

실시간 광고 클릭 집계는 클릭 ID를 키로 쓴다. SDK 재시도나 네트워크 이중 전송으로 같은 클릭이 두 번 들어오면 CTR(Click-Through Rate, 클릭률)이 과대 산출되고, 광고주 정산이 그 수치를 기준으로 하므로 중복 제거 키 설계가 곧 매출 정확도가 된다. 5.1에서 공공의 요구로 설명한 “중복을 계수해 남긴다”가 여기서는 정산 근거라는 이름으로 똑같이 요구된다.

SaaS 이벤트 파이프라인은 로그인·기능 사용 같은 행동 이벤트를 분석 웨어하우스(warehouse)에 적재한다. at-least-once 전달 위에 Upsert를 얹어 exactly-once 반영을 만드는 구조가 실습 5.1의 A→B→C 순서 그대로이고, 문법만 PostgreSQL의 `ON CONFLICT` 나 BigQuery의 `MERGE` 로 바뀐다. 이 절의 지연 이벤트도 함께 수반된다 — 늦게 도착한 전환·환불을 반영해 집계를 다시 쓰려면 구간 키 Upsert가 필요하다.

세 현장을 관통하는 것은 하나다. 중복 제거 키를 무엇으로 할지, 그 키를 언제 발급할지, 충돌을 실패·무시·병합 중 무엇으로 처리할지, 지연을 언제까지 기다릴지 — 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, 근거만 도메인에 따라 달라진다. 공공에서는 감사·형평성·법적 의무가 근거이고, 민간에서는 매출·비용·SLA다. 방향은 비대칭이다. 공공 설계를 민간에 도입할 때는 증거 산출 의무를 제외해도 손실이 없다 — `seen_count` 는 감사 답변 대신 장애 조기 탐지 지표가 되고, 지문 해시는 정산 재현의 근거가 된다.

지연 이벤트는 이벤트 시간과 처리 시간의 차이가 집계에 드러난 것이다.

watermark는 “언제까지 기다릴 것인가”의 선언이며, 정확성과 확정 속도를 맞바꾼다.

지연분 재계산은 구간 키 기반 Upsert와 결합해야 일관성이 유지된다.

잠정치와 확정치의 구분·표기는 공공 보고 신뢰의 기본 장치다.

중복 제거 키·키 발급 시점·충돌 처리·지연 허용값이라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 5.1 중복 이벤트 주입과 제거 검증(★★☆☆☆)

실습 목표

실측 중복 스트림을 세 가지 저장 전략으로 적재해 결과 차이를 검증한다.
전체 스트림 리플레이(재주입)에도 최종 상태가 안정적임을 확인한다.

네 단계는 각각 본문의 명제 하나씩을 검증하도록 설계되어 있다.

단계	검증하는 본문 명제
A. naive INSERT	5.1 “방어선 없는 저장소는 상류 중복을 데이터로 만든다”
B. 유니크 + OR IGNORE	5.3 “유니크 제약은 중복을 원자적으로 거른다”
C. UPSERT	5.3 “병합은 중복을 관찰 데이터로 바꾼다”
D. 전체 리플레이	5.1 “먹등 쓰기가 exactly-once <i>반영</i> 을 만든다”

입력 데이터

입력(`practice/chapter4/data/input/ch4_alo_duplicate_stream.jsonl`)은 4장 실습 4.1 시나리오 C가 실제 Kafka 브로커에서 남긴 처리 기록의 스냅샷이다 — 40건 중 고유 이벤트 30건, 크래시 후 재수신 중복 10건. 즉 이 실습의 “중복 주입”은 시뮬레이터가 만든 것이 아니라 **실제 브로커 장애 실험이 만든 중복**이며, 스냅샷을 동봉했으므로 4장 환경 없이 단독 실행된다.

실행

```
cd practice/chapter4
python3 run_dedup.py
```

추가 설치는 없다(표준 라이브러리 `sqlite3` 사용). 저장소로 SQLite를 사용하는 이유는 두 가지다 — 파이썬에 내장되어 실습 준비 부담이 없고, 유니크 제약·UPSERT라는 이 장의 핵심 개념이 주요 관계형 DB의 공통 패턴이라 PostgreSQL 등 운영 DB로 그대로 옮겨진다(DB별 문법 세부만 다르다 — 5.3 참조). 스크립트는 같은 스트림을 네 단계로 적재·비교한다: A) 제약 없는 naive INSERT, B) 유니크 제약 + INSERT OR IGNORE, C) UPSERT(재관찰 기록), D) C 상태에서 전체 스트림을 한 번 더 리플레이.

핵심 코드

```
cur = conn.execute("INSERT OR IGNORE INTO complaints_unique VALUES (?, ?, ?, ?)", row)
ignored += 1 - cur.rowcount      # rowcount 0 = 유니크 충돌로 무시됨
```

전체 코드는 `practice/chapter4/code/4-3-deduplicate-events.py` 참고

산출물 안내

산출물	내용	용도
ch4_dedup_report.json	단계별 행 수·중복 수·지문 해시	중복 제거 검증 결과 요약 — 검수 자산
ch4_dedup.sqlite	세 전략의 테이블이 공존하는 DB	직접 질의하며 탐구(예: seen_count 분포)
data/input/ch4_alo_duplicate_stream.jsonl	4장 실측 중복 스트림 스냅샷	입력 — 출처가 문서화된 실측 데이터

실행 결과(실제)

검증 결과 요약은 `practice/chapter4/data/output/ch4_dedup_report.json` 에 저장된다.

표 5.6 저장 전략별 검증 결과(실제 실행 — 입력 40건, 고유 30건)

단계	행 수	관찰 지표	해석
A. naive INSERT	40	중복 행 10	상류 중복이 그대로 데이터가 됨
B. 유니크 + OR IGNORE	30	무시된 중복 10	중복 제거, 단 발생 사실은 소실
C. UPSERT	30	총 관찰 40, 재관찰 이벤트 10, 업무 상태 지문 eed1522a7ca844bd	중복 제거 + 발생 기록 보존
D. 전체 리플레이 (재주입)	30 (불변)	총 관찰 80, 업무 상태 지문 eed1522a7ca844bd (C와 동일)	업무 상태는 재처리에 불변 = 멍등 반영

워크드 예제: 한 행의 최종 상태 확인(실측)

검증이 끝난 뒤 UPSERT 테이블에서 5.1절의 그 이벤트(alo-000003)를 조회하면 다음 한 행이 조회된다.

event_id	파티션	오프셋	first_seen_at	last_seen_at	seen_count
alo-000003	0	0	01:59:20	01:59:35	4

이 한 행이 실습 전체를 요약한다. 입력 스트림에서 두 번(크래시 전·재시작 후), D 단계 리플레이에서 또 두 번 — 합계 네 번 도착했지만 행은 하나다. `first_seen_at` 은 최초 도착 시각 (01:59:20)을 보존하고, `last_seen_at` 은 마지막 도착 시각을 가리키며, `seen_count=4` 가 전달 이력을 기록한다. 전체 분포도 실측과 정확히 일치한다: 중복 없던 20건은 `seen_count` 2(원본 1 + 리플레이 1), 중복 10건은 `seen_count` 4(원본 2 + 리플레이 2) — 합계 $20 \times 2 + 10 \times 4 = 80$, 표 5.6의 총 관찰 수와 일치한다.

실행 중 확인 체크리스트

- ☐ A 단계: 행 수 40 — 중복이 행으로 적재되었는가
- ☐ B 단계: 행 수 30, 무시 10 — `cur.rowcount` 가 충돌을 계수했는가
- ☐ C 단계: 행 수 30인데 총 관찰 40 — 중복이 “기록”으로 남았는가
- ☐ D 단계: 행 수 그대로 30, 지문 해시가 C와 동일한가 — 멍등 반영의 증거

관찰

A와 B·C의 행 수 차이(40과 30)가 5.1절의 명제를 실측으로 확인한다: 저장소에 방어선이 없으면 4장의 재수신 중복 10건이 그대로 집계 대상 데이터가 된다.

B와 C는 행 수가 같지만(30) 남기는 정보가 다르다. B는 중복이 “있었다는 사실”까지 제거하고, C는 `seen_count` 로 보존한다 — C에서 재관찰 이벤트 10은 4장 크래시 실험의 재수신 10건과 정확히 일치하며, 저장소 지표만으로 상류 장애를 역추적할 수 있음을 뜻한다.

D가 이 실습의 결론이다. 전체 스트림을 다시 재생해도(4.1절의 리플레이에 해당) **업무 상태는 변하지 않는다** — 행 수 30 불변에 더해, 업무 필드(event_id·파티션·오프셋·최초 관찰 시각)만으로 만든 지문 해시가 C와 D에서 동일하다(eed1522a7ca844bd). at-least-once 전달 위에서 멍등 쓰기가 만들어 내는 “정확히 한 번 반영”이란 정확히 이 의미, 즉 **업무 상태의 리플레이 불변성**이다.

단, 정밀하게 구분할 것: 전달 관찰 메타데이터(`seen_count` · `last_seen_at`)는 리플레이에서 40→80으로 **의도적으로 증가한다**(전달 횟수의 기록이므로). 따라서 `seen_count` 를 상류 건강 지표로 사용할 때는 장애 재수신과 계획된 리플레이(재처리)를 구분해서 해석해야 하며, 구분하지 않으면 지표가 과대 산출된다. “무엇이 업무 상태이고 무엇이 관찰 메타데이터인가”를 스키마 설계 때 정해 두는 것이 이 구분의 출발점이다.

B의 무시된 중복 10건은 4장 관찰 보고서의 재수신 중복 10건과 정확히 같은 이벤트들이다 — 두 장의 산출물이 같은 사실을 서로 다른 위치(브로커 소비 기록과 저장소 적재 통계)에서 확인하며, 이런 교차 검증 가능성 자체가 관찰 가능한 파이프라인의 가치다.

이 실습이 격리해 보인 UPSERT 흡수는 14장 통합 MVP에서 부품이 이어진 상태로 재현된다 — 재전송 드릴에서 물리 수신 28건이 고유 17건으로 축약되고(중복 흡수 11건 = 재전송 10 + 원천 중복 접수 1) 매핑 집계는 7장 확정치 16건으로 불변이었다. 5장이 SQLite 테이블 하나에서 증명한 멍등 반영이, 14장에서는 Kafka·프로세서·집계가 연결된 흐름 전체에서 같은 원리로 성립한다. 소규모 실습의 결론이 통합 규모에서도 유지됨을 확인하는 회수 지점이다. 이 실습의 중복은 모두 “내용이 동일한 재전송”이므로 무시(B)로도 충분했다. 재전송이 갱신을 담는 시스템이라면 병합(C)이 필수가 된다(5.3절의 선택 기준).

위험 식별(검증 결과 연계)

위험	실습에서의 근거	대응 방향
무방비 저장소의 통계 왜곡	A: 중복 행 10이 그대로 적재	중복 제거 키 + 유니크 제약을 스키마에
기술 키가 업무 중복까지 제거한다는 오해	5.2 워크드(재접수는 통과가 정상)	업무 규칙 계층 분리, 자동 삭제 금지
중복 발생 사실의 소실	B: 무시 10건이 흔적 없이 소실됨	병합(seen_count)으로 관찰 지표화
재처리·리플레이 시 상태 오염	D: 병합 설계에서만 행 수 불변 확인	재처리 경로도 멍등 쓰기로 통일

보충(전문가 트랙 포인터)

Kafka 트랜잭션과 exactly-once semantics: 소비-변환-생산 경로를 트랜잭션으로 묶어 스트림 처리 내부의 exactly-once를 만드는 방식. 단, 그 보장은 Kafka 생태계 안으로 한정된다 — 최종 저장소가 Kafka 바깥(이 실습처럼 관계형 DB)이면 이 장의 멍등 쓰기가 여전히 필요하다. “exactly-once 지원”이라는 제품 문구에서 보장의 경계가 어디까지인지 묻는 것이 검수 포인트다. 이 지점에는 유명한 논쟁이 있다 — 한쪽은 “exactly-once 전달은 불가능하다”(Tyler Treat, 2015)라 하고, 다른 쪽은 “exactly-once는 가능하다”(Confluent, Narkhede 2017)라 한다. 모순처럼 보이지만 두 주장은 서로 다른 대상을 말한다: 전자는 임의의 두 노드 사이의 전달을, 후자는 멍등 Producer와 트랜잭션으로 만드는 Kafka 내부의 처리 반영을 가리킨다. 5.1에서 세운 “전달 보장인가 반영 보장인가”의 구분이 두 주장의 차이를 정확히 설명한다(둘 다 배경 자료).

Redis 기반 dedup cache의 TTL 설계: 유니크 제약 대신(또는 DB 부하를 줄이기 위해 앞단에) 인메모리 캐시로 중복을 거르는 방식. 캐시 항목은 TTL(만료 시간)로 정리되는데, TTL이 중복 발생 가능 시간창(예: 재시도 최대 지연)보다 짧으면 만료 후 도착한 중복이 통과한다 — watermark와 같은 구조의 “언제까지 기억할 것인가” 문제다.

감사 추적과 삭제 불가능 로그의 긴장: 감사 로그는 “삭제하지 않는 것”이 원칙이라 중복 제거와 충돌한다. 원본 로그(불변)와 정제 테이블(중복 제거)을 분리하는 것이 통상적 해법이며, 실습의 A(원본 그대로)와 C(정제+관찰)를 나란히 두는 구조가 그 축소판이다.

이 장이 남기는 검수 자산(부록 D 연계)

중복 시나리오별 대응표(표 5.1): 단계별 발생 경로와 대응의 단일 정리

키 설계 점검 체크리스트(5.2): 중복 제거 키 확정 전 점검 항목

중복 제거 검증 결과 요약(ch4_dedup_report.json): 전략별 실측 비교 — 지문 해시로 멍등 반영까지 증빙

핵심 용어

중복 제거 키: “같은 이벤트”를 판정하는 기준 필드 — 재시도·재처리 경로에서 불변이어야 한다

유니크 제약: 저장소가 원자적으로 지키는 무중복 방어선 — 코드의 확인-후-저장과 달리 경쟁 조건에 뚫리지 않는다

Upsert: 충돌 시 기존 행을 갱신하는 병합형 INSERT — 무엇을 갱신·보존할지 SQL에 명시된다

멥등 쓰기: 같은 이벤트를 여러 번 반영해도 최종 상태가 같은 쓰기 — 엘리베이터 버튼을 반복해서 눌러도 호출이 하나로 유지되는 것과 같은 구조

멥등성 키(idempotency key): 요청·이벤트 생성 시점에 발급되어 재시도 경로에서 불변인 고유 ID — Stripe 결제 API의 산업 표준 구현(중복 제거 키의 클라이언트 발급 형태)

경쟁 조건(race condition): 확인과 기록 사이의 간격에 다른 실행이 삽입되어 발생하는 오동작

seen_count(재관찰 기록): 중복을 버리는 대신 전달 이력으로 남기는 병합 설계의 관찰 필드

watermark: 지연 이벤트를 언제까지 기다릴지의 선언(6장에서 구현)

다음 장 예고

5.4절은 지연 이벤트를 개념으로만 열어 두었다 — “언제까지 기다릴 것인가”라는 질문에 watermark라는 선언으로 답한다는 것까지. 6장은 그 선언을 실제 스트리밍 엔진(Spark Structured Streaming)에서 구현한다. 지역별 민원 이벤트의 윈도우 집계를 만들고, 지연 이벤트 두 건을 의도적으로 투입해 하나는 수용되고(집계가 갱신되고) 하나는 폐기되는(카운터만 남는) 것을 실측으로 관찰한다. 그리고 그 집계 출력이 다시 이 장의 Upsert를 필요로 한다는 것 — 두 장이 하나의 고리로 연결되는 것을 확인한다.

참고문헌

Apache Kafka. (n.d.). *Documentation: Message Delivery Semantics*.

<https://kafka.apache.org/documentation/#semantics>

SQLite. (n.d.). *UPSERT*. https://sqlite.org/lang_upsert.html

SQLite. (n.d.). *ON CONFLICT clause*. https://www.sqlite.org/lang_conflict.html

Python Software Foundation. (n.d.). *sqlite3 — DB-API 2.0 interface for SQLite databases*.

<https://docs.python.org/3/library/sqlite3.html>

Leach, B. (Stripe). (2017). *Designing robust and predictable APIs with idempotency*.

<https://stripe.com/blog/idempotency> (적용 자격: 직접 선례 — 결제 중복 방지의 멥등성 키 설계)

Stripe. (n.d.). *Idempotent requests* (API Reference).

https://docs.stripe.com/api/idempotent_requests (적용 자격: 직접 선례 — Idempotency-Key 헤더 규격)

Treat, T. (2015). *You Cannot Have Exactly-Once Delivery*. <https://bravenewgeek.com/you-cannot-have-exactly-once-delivery/>

(적용 자격: 배경 자료 — 저자 입장이 뚜렷한 블로그 글. 핵심 논거는 Two Generals 사고실험(ACK 불확실성)이며, FLP는 합의 한계를 보이는 관련

배경으로 언급된다. FLP 자체는 전달이 아니라 비동기 합의의 종료 불가능성 결과임에 유의)
PostgreSQL Global Development Group. (n.d.). *INSERT — ON CONFLICT clause*.

<https://www.postgresql.org/docs/current/sql-insert.html> (적용 자격: 방법론 차용 — SQLite
실습의 운영 DB 이식)

Narkhede, N. (Confluent). (2017). *Exactly-once Semantics is Possible: Here's How Apache Kafka Does it*. <https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/> (적용 자격: 배경 자료 — exactly-once 논쟁의 반대편. Kafka 내부의 멍든 Producer-트랜잭션으로 만드는 처리 보장을 다룸)